

系統架構文件

Sync104ToBpmErp

104 HR Max → BPM (電子簽核) / ERP (TIPTOP) 資料同步程式

版本 v1.0 | 撰寫日期 2026-07-12

範圍 : D:\Project\Sync104ToBpmErp (原始碼實作現況)

1. 系統概述

Sync104ToBpmErp 是一支 .NET 8.0 主控台 (Console) 程式，負責將 104 HR Max (104人力銀行 HR API) 的組織與人員資料，定期同步到企業內部兩套系統：

目標系統	資料庫	說明
BPM	MS-SQL (EFGP 電子簽核系統)	寫入 OrganizationUnit 、 OrganizationUnitLevel 、 Users 、 Employee
ERP	Oracle (TIPTOP)	寫入 YCS.GEM_FILE (部門) 、 YCS.ABD_FILE (部門層級關係) 、 YCS.GEN_FILE (員工)

程式本身不提供常駐服務，而是設計為由 Windows 工作排程器定期呼叫，每次執行處理一段查詢時間範圍內的異動資料，執行完畢即結束 (Console Exit) 。

2. 技術架構

項目	技術 / 版本
開發框架	.NET 8.0 (Sync104ToBpmErp.csproj)
ORM	Dapper 2.1.72
MS-SQL 連線	Microsoft.Data.SqlClient 7.0.1
Oracle 連線	Oracle.ManagedDataAccess.Core 23.26.200
設定管理	Microsoft.Extensions.Configuration (appsettings.json + 環境變數)
日誌	Serilog 4.3.1 (Console + 檔案雙輸出，依日期滾動)
HR API 呼叫	HttpClient (JSON / POST)

3. 專案結構

```
Sync104ToBpmErp/
├─ Program.cs                # 進入點：命令列參數解析、互動模式、四大執行分支
├─ appsettings.json          # 執行環境設定 (HR API、BPM/ERP 連線字串、同步參數)
├─ Configuration/
│   └─ AppSettings.cs        # HRApiSettings / DatabaseSettings / SyncSettings
├─ Models/                   # 104 API 回應模型、OID 產生器、同步結果模型
│   ├── CompanyInfo.cs / Department.cs / Employee.cs / DeptHierarchy.cs
│   ├── ApiResponse.cs / ApiTokenResponse.cs
│   ├── OidHelper.cs          # BPM OID (32碼) 產生器
│   └─ SyncResult.cs
├─ Services/
│   ├── HRApiService.cs       # 呼叫 104 HR Max API (Token、公司、部門、部門層級、員工)
│   ├── BpmDatabaseService.cs # 寫入 BPM (MS-SQL)
│   ├── ErpDatabaseService.cs # 寫入 ERP (Oracle)
│   ├── SyncService.cs        # 同步流程協調 (Orchestrator) 與同步報告產生
│   └─ LoggerService.cs       # Serilog 封裝 (SyncLog / ErrorLog)
├─ SQL/                      # BPM / ERP 資料表結構參考腳本
├─ DOC/                      # 104 API 文件、欄位對照草案、部門/公司示意圖
├─ deploy.bat / package.ps1  # 發布打包腳本
└─ 部署說明.md / README.md   # 部署與使用說明 (既有文件)
```

4. 執行參數總覽 (Program.cs → Main)

程式進入點依 `args[0]` 判斷執行分支，不分大小寫：

參數	別名	需要額外參數	功能	是否連線資料庫
(無參數)	—	互動輸入	互動模式：提示輸入起訖時間後執行完整同步	是
<code>--help</code>	<code>-h</code> 、 <code>/?</code>	無	顯示使用說明後結束	否
<code>--test-connection</code>	<code>-t</code>	無	測試 BPM、ERP、HR API 三項連線	是 (僅測試)
<code>--test-api</code>	<code>-a</code>	無	僅測試 HR API (不連資料庫，不寫入任何資料)	否
<code>--sync</code>	<code>-s</code>	需 2 個參數：開始時間、截止時間	執行完整同步 (可供排程器呼叫)	是
其他未知字串	—	—	顯示「未知的參數」提示後結束 (Exit，不執行同步)	否

時間參數格式 (`TryParseDateTime`，依序嘗試)：1. `yyyy-MM-dd HH:mm:ss` (完整時間) 2. `yyyy-MM-dd` (自動補 `00:00:00`) 3. .NET 預設 `DateTime.TryParse` 格式 (保底)

若格式皆解析失敗，或截止時間早於開始時間，程式印出錯誤訊息並以 **Exit Code 1** 結束。

5. 各參數執行流程詳解

5.1 Sync104ToBpmErp.exe --help / -h / /?

```
Main() → 印出 Banner → 解析 args[0]="--help" → ShowHelp() → return (Exit 0)
```

僅輸出使用說明文字（用法、時間格式、Log 檔案位置），不載入任何資料庫連線，不呼叫 HR API。

5.2 Sync104ToBpmErp.exe --test-connection / -t

```
Main() → TestConnectionsAsync(settings, logger)
├─ 1. new BpmDatabaseService → TestConnectionAsync() → 開啟 MS-SQL 連線並印出成功/失敗
├─ 2. new ErpDatabaseService → TestConnectionAsync() → 開啟 Oracle 連線並印出成功/失敗
└─ 3. new HRAPIService → GetAccessTokenAsync() → 呼叫 AuthEndpoint 取得 Token
→ 印出三項測試結果 → return (Exit 0，不論成功失敗都不改變 Exit Code)
```

用途：部署後第一次執行前的健檢，不寫入任何資料。

5.3 Sync104ToBpmErp.exe --test-api / -a

```
Main() → TestHRApiAsync(settings, logger)
├─ 1. 取得 Access Token
├─ 2. 呼叫 /api/os/company 取得公司清單 (CO_ID / CO_CODE / CO_NAME)
├─ 3. 設定時間範圍 = 最近 365 天 (endTime=Now, startTime=Now-365)
└─ 4. for each company:
    ├─ 3.1 呼叫 EmployeeEndpoint 測試取得員工資料
    ├─ 3.2 呼叫 DepartmentEndpoint 測試取得部門資料
    └─ 3.3 呼叫 HierarchyEndpoint 測試取得部門層級資料
→ 彙總印出「所有測試通過」或「部分測試失敗」
```

用途：驗證 104 API 端點、帳密、公司清單是否可正常取得，完全不連 BPM/ERP 資料庫，也不寫入資料。任一項失敗只記錄 Log 並繼續下一項，不會中斷整體測試。

5.4 Sync104ToBpmErp.exe --sync "起始時間" "結束時間" / -s

```
Main() → 檢查 args.Length >= 3?
├─ 否 → 印出「需要提供開始/截止時間」用法提示 → return (不執行同步)
└─ 是 → RunSyncWithArgsAsync(settings, logger, args)
    ├─ 解析 args[1] 為 startTime (失敗 → Exit 1)
    ├─ 解析 args[2] 為 endTime (失敗 → Exit 1)
    ├─ 檢查 endTime >= startTime (否則 Exit 1)
    └─ ExecuteSyncAsync(settings, logger, startTime, endTime)
        → 建立 HRAPIService / BpmDatabaseService / ErpDatabaseService
        → new SyncService(...).RunFullSyncAsync(startTime, endTime)
        → 印出文字報告 (GenerateTextReport)
        → Environment.Exit(report.Success ? 0 : 1)
```

此模式不需人工互動，是 Windows 工作排程器唯一應使用的執行方式。

5.5 互動模式 (不帶任何參數執行)

```
Main() → args.Length == 0 → RunInteractiveModeAsync(settings, logger)
├─ 迴圈提示輸入「開始時間」直到格式正確
├─ 迴圈提示輸入「截止時間」直到格式正確且 >= 開始時間
├─ 印出確認畫面，等待輸入 Y/YES 才繼續 (其餘輸入視為取消)
└─ 確認後 → ExecuteSyncAsync(...) (與 5.4 相同的同步核心)
```

用途：人工手動測試同步，第一線人員部署驗證時常用。

5.6 未知參數

```
args[0] 不符合上述任何 case → 印出「未知的參數: {args[0]}」+「使用 --help 查看說明」→ return
```

四種進入分支的共同前置流程：不論哪個分支，`Main()` 一開始都會先：1. 關閉 Oracle ODP.NET 的 TNS 名稱解析 (避免無 `tnsnames.ora` 時 `EzConnect` 被誤判)；2. 載入 `appsettings.json` → 綁定為 `AppSettings` (找不到檔案或綁定失敗直接 `Exit 1`)；3. 依 `SyncSettings.LogDirectory` 初始化 `LoggerService` (`Serilog`)。

6. 核心同步引擎： `SyncService.RunFullSyncAsync(startTime, endTime)`

只有 5.4 (命令列 `--sync`) 與 5.5 (互動模式) 兩種進入點會執行到這裡；`--test-connection`、`--test-api`、`--help` 皆不會觸碰此流程。

```
RunFullSyncAsync(startTime, endTime)
├─ 0. TestConnectionsAsync()
│   ├── BPM 連線失敗 → 拋例外，整體同步中止 (Success=false)
│   └─ ERP 連線失敗 → 拋例外，整體同步中止 (Success=false)
├─ 1. GetCompaniesAsync() → /api/os/company
│   ├── 公司清單僅供「查詢比對」，公司資料本身不寫入 BPM Organization 或 ERP geu_file
│   ├── (兩表皆假設已由系統管理端事先建立)
│   └─ 若公司清單為空 → 視為成功但無資料可同步，提早結束
├─ 2. for each company (依 CO_ID/CO_CODE 迴圈):
│   ├── 2.1 部門層級名稱同步 SyncDeptHierarchyAsync(coId, coCode)
│   │   ├── 104 /api/os/dept_level → BPM OrganizationUnitLevel
│   │   └─ (無 ERP 對應動作，ERP abd_file 改由 2.2 從 Department.ParentDeptCode 產生)
│   ├── 2.2 部門資料同步 SyncDepartmentsAsync(startTime, endTime, coId, coCode)
│   │   ├── 104 /api/os/dept → BPM OrganizationUnit
│   │   │   ├── ERP YCS.GEM_FILE
│   │   │   └─ ERP YCS.ABD_FILE (由部門的 ParentDeptCode 推導父子關係)
│   └─ 2.3 員工資料同步 SyncEmployeesAsync(startTime, endTime, coId)
│       ├── 104 /api/ed/emp → BPM Users (先寫) + Employee (後寫，需 Users.OID/OrganizationUnit.OID)
│       └─ ERP YCS.GEN_FILE
```

同步順序具強制相依性：部門層級 → 部門 → 員工。

任一步驟找不到資料 (104 回傳空陣列) → 記錄 `Warning` 並跳過，不中斷整支排程。

任一步驟拋出例外 → 整個 `RunFullSyncAsync` 標記 `Success=false`，記錄錯誤訊息，`finally` 區塊仍會產出含耗時的結束 `Log` 與 `SyncReport`。

時間範圍參數的作用範圍：`startTime` / `endTime` 只套用在「部門」（`E_SDATETIME` / `E_EDATETIME` 維護日期）與「員工」（`C_SDATETIME` / `C_EDATETIME` 新增日期、`E_SDATETIME` / `E_EDATETIME` 維護日期）兩支 API；「公司清單」與「部門層級名稱」API 呼叫不帶時間區間參數，每次執行都會取得當下全量清單。

執行結果：`SyncService` 回傳 `SyncReport`，內含依 `CO_ID` 分組的 BPM / ERP 各表新增/跳過/失敗筆數與錯誤明細（取前 20 筆），`Program.cs` 將其轉為文字印出到主控台，並以 `report.Success` 決定 **Exit Code**（0=成功，1=失敗），供排程器判斷本次執行是否成功。

7. 資料寫入邏輯（現行實作重點）

7.1 現行為「INSERT-or-SKIP」，尚未真正 UPDATE

程式碼中每張表的寫入方法都遵循相同模式：依業務鍵查詢是否已存在 → 已存在則記錄「SKIP(已存在)」並跳過 → 不存在則 INSERT。

⚠ 重要現況：`BpmDatabaseService.cs`、`ErpDatabaseService.cs` 內對應的 UPDATE 陳述式目前皆以註解方式保留在程式碼中（例如 `OrganizationUnit`、`OrganizationUnitLevel`、`Users`、`Employee`、`gem_file`、`abd_file`、`gen_file`），尚未實際執行。也就是說，104 端資料異動後若在 BPM/ERP 已存在對應紀錄，目前不會覆蓋更新，只會被略過並計入 `SkippedCount`。日後若需要真正的 UPSERT（含更新既有資料），需取消這些區塊的註解並補齊測試。

7.2 OID 產生規則（BPM 專用）

`Models/OidHelper.cs` 使用 `Guid.NewGuid().ToString("N")` 產生 32 碼小寫十六進位字串作為 OID（非隨機英數混合），並透過 `GenerateUniqueAsync` 對指定的多張表（依表而定，如 `OrganizationUnit`、`OrganizationUnitLevel`、`Employee`、`Users`）逐一檢查是否重複，最多重試 10 次。

7.3 部門拓撲排序（Topological Sort）

`BpmDatabaseService.TopologicalSortDepartments()` 會依 `ParentDeptCode` 對部門清單做拓撲排序，確保父部門一定排在子部門之前寫入，讓子部門查詢 `superUnitOID` 時能查到剛寫入的父部門 OID（同一批次內的記憶體對照表 `orgUnitOidMap` 會即時更新，避免額外資料庫查詢）。

7.4 交易與批次處理

每個 `SyncXxxAsync` 方法（部門、部門層級、員工）各自開啟一個資料庫連線 + 一個交易（Transaction），處理該批次全部資料後統一 `Commit()`；若中途拋出未捕捉例外則整批 `Rollback()`。單筆資料的例外會在迴圈內被攔截（記入 `FailedCount` 與 `Errors`），不會導致整批回滾，只有連線層級或非預期例外才會讓整批交易失敗。`SyncSettings.BatchSize`（預設 100）目前僅用於每處理 100 筆印一次進度 Log，尚未用於實際分批寫入或分批查詢。

7.5 關聯查詢與 N+1 問題的處理

`SyncOrganizationUnitsAsync` 在交易開始時會一次性把 `Users`、`OrganizationUnit`、`OrganizationUnitLevel` 全表查入記憶體字典，後續逐筆比對皆走記憶體查詢，避免每筆部門都各自下一次 SQL（N+1 問題）。`SyncEmployeesAsync` 則是逐筆即時查詢 `Users.OID`、`OrganizationUnit.OID`、`Employee.OID`（未做批次預載）。

8. 資料對照總表 (104 → BPM / ERP)

104 API	資料型態	BPM 目標表	ERP 目標表	業務主鍵 (UPSERT 判斷欄位)
<code>/api/os/company</code> (<code>CompanyEndpoint</code>)	公司	不寫入 (假設已建立)	不寫入 (假設已建立)	—
<code>HierarchyEndpoint</code> (預設 <code>/api/os/dept_level</code>)	部門層級名稱	<code>OrganizationUnitLevel</code>	不寫入	<code>levelValue</code> + <code>organizationOID</code>
<code>DepartmentEndpoint</code> (預設 <code>/api/os/dept</code>)	部門	<code>OrganizationUnit</code>	<code>YCS.GEM_FILE</code> (<code>GEM01</code> = 部門代碼) 、 <code>YCS.ABD_FILE</code> (<code>ABD01</code> = 父層代碼 、 <code>ABD02</code> = 自身代碼)	<code>id</code> (BPM) / <code>GEM01</code> (ERP)
<code>EmployeeEndpoint</code> (預設 <code>/api/ed/emp</code>)	員工	<code>Users</code> (先寫) + <code>Employee</code> (後寫)	<code>YCS.GEN_FILE</code> (<code>GEN01</code> = 員工編號)	<code>id / employeeId</code> (BPM) / <code>GEN01</code> (ERP)

8.1 部門 → BPM `OrganizationUnit` (節錄重點欄位)

BPM 欄位	來源	說明
<code>OID</code>	<code>GenerateUniqueOIDAsync()</code>	新增時產生，查表確保不重複
<code>id</code>	104 <code>DEPT_CODE</code>	部門代碼
<code>organizationUnitName</code>	104 <code>DEPT_NAME</code>	部門名稱
<code>managerOID</code>	查 <code>Users</code> (依 104 <code>LEADER_EMP_NO</code>)	需員工已先同步
<code>superUnitOID</code>	查 <code>OrganizationUnit</code> (依 104 <code>PARENT_DEPT_CODE</code>)	依拓撲排序確保父部門已寫入
<code>levelOID</code>	查 <code>OrganizationUnitLevel</code> (依 <code>DeptLevelId</code> + <code>organizationOID</code>)	需部門層級已先同步
<code>organizationOID</code>	查 <code>Organization</code> (依公司 <code>CO_CODE</code>)	找不到則寫入 <code>NULL</code> 並記 Warning
<code>objectVersion</code>	固定 1	目前僅支援新增，尚無更新版次遞增邏輯
<code>validType</code>	依 104 <code>IS_ACT</code> (1→1，其餘→0)	

8.2 部門 → ERP `YCS.GEM_FILE` (節錄)

欄位	來源	現行寫入值
<code>GEM01</code> (PK)	104 <code>DEPT_CODE</code>	
<code>GEM02</code>	104 <code>DEPT_NAME</code>	

欄位	來源	現行寫入值
GEM03	DeptAbbr 或 DeptName	
GEMACTI	104 IS_ACT	1→ Y · 其餘→ N
GEMUSER / GEMMODU / GEMORIU	固定值	SYNC104
GEMDATE	系統時間	SYSDATE
GEM04/05/06/07/08/09/10/11 、 GEMGRUP 、 GEMORIG	—	目前皆寫入 NULL (欄位用途待客戶確認)

8.3 部門層級關係 → ERP YCS.ABD_FILE

資料來源並非 HierarchyEndpoint (dept_level API)，而是改由部門清單的 ParentDeptCode 推導：- ABD01 = 父層部門代碼 (無父層則等於自己)，取前 10 碼 - ABD02 = 自身部門代碼，取前 10 碼 - ABDACTI 固定 Y；ABDUSER / ABDMODU / ABDORIU 固定 SYNC104

ErpDatabaseService.SyncAbdFileAsync(List<DeptHierarchy>) (對應 dept_level API 版本) 目前為保留介面但不實作，僅記錄 Log 說明已改用部門版本。

8.4 員工 → BPM Users + Employee

Users (先寫)：id = EMP_NO、userName = EMP_NAME、password = SyncSettings.DefaultPassword (預設 "0000")、mailAddress = OFFICE_EMAIL、phoneNumber = OFFICE_TEL、leaveDate = QUIT_DATE，其餘系統欄位 (identificationType = Employee、localeString = zh_TW、enableSubstitute = 0 等) 皆為固定值。

Employee (後寫，需先有 Users.OID)：employeeId = EMP_NO、organizationOID = 依 104 DEPT1_CODE 查 OrganizationUnit.id、userOID = 依 EMP_NO 查剛寫入的 Users.OID、validTo = QUIT_DATE。

注意：104 員工資料實際支援 5 層部門 (DEPT1_CODE ~ DEPT5_CODE)，但目前程式僅使用第一層 DEPT1_CODE 作為員工所屬部門，向下相容屬性 DeptCode 即為 Dept1Code 的別名。

8.5 員工 → ERP YCS.GEN_FILE

欄位	來源	現行寫入值
GEN01 (PK)	104 EMP_NO	
GEN02	104 EMP_NAME	
GENACTI	104 WORK_STATUS (=3 判定離職)	離職→ N · 其餘→ Y
GENUSER / GENMODU	固定值	SYNC104
GEN03 (部門)、GEN04 (職稱)、 GEN05、GEN06 (Email)、 GENGRUP、TA_GEN07、TA_GEN08	—	目前皆寫入 NULL (欄位對應待客戶確認，DOC/程式邏輯.md 為既有規劃草案)

9. Log 記錄機制

`LoggerService` (Serilog) 在每次程式啟動時依 `SyncSettings.LogDirectory` (預設 `Logs/` , 相對路徑會轉為以執行檔所在目錄為基準) 建立兩條輸出管線 :

檔案	層級	內容	保留策略
<code>Logs/SyncLog_yyyyMMdd.txt</code>	Information 以上	程式啟動、連線測試、API 呼叫筆數、同步進度 (每 100 筆)、同步完成統計	依日期滾動, 保留最近 30 個檔案
<code>Logs/ErrorLog_yyyyMMdd.txt</code>	Error 以上	連線失敗詳情、單筆同步失敗明細 (含資料 Key 與錯誤訊息)、交易回滾事件	同上

同時輸出到 Console , 方便排程器擷取執行輸出或人工即時觀察。成功的單筆同步僅記錄於 **Debug 層級** (不會寫入上述兩個檔案) , 避免大量資料同步時 Log 檔暴增。

10. 設定檔說明 (`appsettings.json`)

```
{
  "HRApi": {
    "BaseUrl": "https://<104 HR Max API 網域>",
    "AuthEndpoint": "/api/auth/signIn",          // 取得 Access Token
    "CompanyEndpoint": "/api/os/company",        // 取得公司清單
    "EmployeeEndpoint": "/api/ed/emp",           // 取得員工資料
    "DepartmentEndpoint": "/api/os/dept",        // 取得部門資料
    "HierarchyEndpoint": "/api/os/dept_level",    // 取得部門層級名稱
    "UserAccount": "<API 帳號>",
    "UserPassword": "<API 密碼>",                // 機敏資訊, 不應外流/進版控
    "OrgTypeCode": "1",                          // 組織類別代碼, 呼叫 dept / dept_level 用
    "BaseDate": "2025/05/07"                     // 基準日, 呼叫 dept / dept_level 用 (需視情況調整)
  },
  "BpmDatabase": {
    "ConnectionString": "Server=<MS-SQL 主機>;Database=<DB>;User Id=<帳號>;Password=<密碼>;TrustServerCertificate=True;",
    "Provider": "MSSQL"
  },
  "ErpDatabase": {
    "ConnectionString": "User Id=<帳號>;Password=<密碼>;Data Source=<主機>:1521/<Service>"
  },
  "SyncSettings": {
    "BatchSize": 100,                            // 目前僅影響進度 Log 頻率
    "LogDirectory": "Logs",
    "SyncIntervalMinutes": 60,                   // 目前未被程式邏輯實際讀取使用 (保留供未來常駐/排程模式)
    "DefaultPassword": "0000"                   // 新建 BPM Users 帳號的預設密碼
  }
}
```

安全提醒 : 目前 `appsettings.json` 內直接以明碼儲存 HR API 帳密與兩套資料庫連線字串 (含密碼) , 部署到客戶環境時務必以檔案系統權限限制存取 , 並避免將含真實密碼的設定檔提交到版本控制。

`HRApiSettings.CompanyId` 欄位雖仍存在於 `Configuration/AppSettings.cs`，但目前同步流程已改為完全依 `/api/os/company` 動態取得公司清單並逐一 Loop，此設定值已無實際作用（保留為未來 fallback 用途）。

11. 部署與排程建議

11.1 編譯與封裝

```
dotnet build
dotnet publish -c Release -o .\publish --self-contained false
```

`deploy.bat` 會自動清理舊產物、發布、複製執行檔 / 設定檔 / SQL 腳本 / 說明文件，並打包為帶日期戳記的 zip（如 `deploy/Sync104ToBpmErp_部署包_20260712.zip`），內含 `Logs/` 空目錄。

11.2 首次部署驗證順序（建議）

1. 在客戶 BPM / ERP 資料庫執行 `SQL/BPM_Schema.sql`、`SQL/ERP_Schema.sql`（若表尚未建立）
2. 依客戶環境修改 `appsettings.json`
3. 執行 `Sync104ToBpmErp.exe --test-connection` 確認三項連線皆成功
4. 執行 `Sync104ToBpmErp.exe --test-api` 確認 104 API 可正常取得公司/部門/員工資料（不落地）
5. 以互動模式或極短時間範圍（如單日）執行一次 `--sync` 測試同步，檢查 `Logs/` 內容
6. 確認無誤後設定 Windows 工作排程器，改用 `--sync` 命令列參數定期執行

11.3 Windows 工作排程器設定範例

設定項目	建議值
程式	<code>C:\Sync104ToBpmErp\Sync104ToBpmErp.exe</code>
引數	<code>--sync "{前一次執行時間} 00:00:00" "{今日} 23:59:59"</code>
觸發頻率	依需求（例如每日凌晨、或每小時）

排程呼叫必須帶滿 `--sync <開始時間> <截止時間>` 兩個參數，否則程式僅印出用法說明並直接結束，不會執行任何同步（見 5.4）。排程器應以程式的 **Exit Code**（0=成功 / 1=失敗）判斷是否需要告警或重試。

12. 錯誤處理與交易機制摘要

- **API 層**：104 API 呼叫失敗（例外）會直接向上拋出，中止當次公司或當支 API 的同步（視呼叫層級而定），並寫入 `ErrorLog`；104 回傳空陣列則視為正常情境（跳過並記 `Warning`，不算錯誤）。
- **資料庫層**：每張表的同步方法各自使用獨立交易；單筆例外只累計失敗筆數並繼續下一筆，非預期例外（如連線中斷）才會讓整批 Rollback 並向上拋出，導致該公司後續步驟（部門/員工）不會執行、但**不影響下一家公司的同步**（`RunFullSyncAsync` 的 `foreach company` 沒有針對個別公司的 `try/catch`，因此目前若某公司同步中拋例外，會直接中止整個 `RunFullSyncAsync`，其後尚未處理的公司也不會執行）。
- **回傳碼**：`Program.Main` 以 `Environment.Exit(report.Success ? 0 : 1)` 結束整支程式，供排程系統/監控工具判讀。

13. 已知限制與待確認事項

1. **僅新增、未更新 (見 7.1)** : 所有目標表的 UPDATE 邏輯已寫成程式碼但被註解停用, 104 端資料異動目前不會回寫覆蓋既有紀錄。
2. **公司迴圈無隔離** : `RunFullSyncAsync` 對多家公司的 `foreach` 迴圈中, 任一公司同步拋出未捕捉例外會中止整個排程, 尚未同步的公司會被跳過 (非逐公司獨立 try/catch)。
3. **`SyncIntervalMinutes` 未被使用** : 設定檔保留此欄位, 但目前程式為單次執行後即結束, 排程頻率完全依賴外部 Windows 工作排程器, 非程式內建計時器。
4. **多筆待確認欄位** : ERP `GEM_FILE` (`GEM04/05/06/07/08/09/10/11` 、 `GEMGRUP` 、 `GEMORIG`) 、 `GEN_FILE` (`GEN03~06` 、 `GENGRUP` 、 `TA_GEN07/08`) 、 BPM `OrganizationUnit.organizationUnitType / validType` 對應規則等, 目前寫入固定值或 `NULL` , 需與客戶 ERP/BPM 管理端進一步確認正式對應規則 (詳見 `DOC/程式邏輯.md` 、 `DOC/api_erp_bpm_mapping.md` 既有規劃草案)。
5. **員工僅取第一層部門** : 104 員工資料含 5 層部門欄位 (`DEPT1_CODE ~ DEPT5_CODE`) , 現行同步僅使用 `DEPT1_CODE` 。
6. **`BatchSize` 未實際分批** : 僅用於進度 Log 顯示頻率, 資料庫寫入仍是單一大交易逐筆執行, 資料量極大時需留意交易時間與鎖定範圍。
7. **敏感資訊明碼儲存** : `appsettings.json` 內 HR API 密碼與兩套資料庫連線密碼皆為明碼, 部署與版控時需額外防護 (見 10 節安全提醒)。